PROMETEO · by thePower

SISTEMAS INFORMÁTICOS - DAM · DAM

Unidad 1: El sistema microinformático: componentes, periféricos y arranque

UNIDAD 1

Índice de contenidos

Objetivos de la unidad 3

Introducción 4

1. Componentes y periféricos 4

1.1. Componentes de un sistema informático

1.2. Periféricos, adaptadores y seguridad

1.3. El proceso de arranque: POST y BIOS/UEFI

1.4. Estructura interna del procesador

1.5. Interrupciones: la base de la multitarea y de la entrada/salida

1.6. La memoria interna: tipos, características y jerarquía

1.7. La placa base: formatos, componentes y buses

1.8. Interfaces de entrada/salida y criterios de elección

2. Introducción a las redes 18

2.1. Características, tipos, componentes y topologías

2.2. Cableado, conectores y mapa físico/lógico

Ampliación y actualización 22

Mini proyecto guiado 23

Ejercicio 24

Resumen profesionalizante 24

RESULTADOS DE APRENDIZAJE

Objetivos de la unidad

Al finalizar esta unidad, serás capaz de:

- Identificar los componentes de un sistema informático.
- Reconocer los periféricos y adaptadores de conexión.
- Aplicar las normas de seguridad y prevención de riesgos.
- Comprender las características, tipos y topologías de las redes.
- Elaborar el mapa físico y lógico de una red local.

ACTIVACIÓN Y REFLEXIÓN

Introducción

Preguntas para reflexionar:

- ¿Qué componentes forman un ordenador y qué hace cada uno?
- ¿Cómo se conectan los equipos entre sí en una red?
- ¿Por qué un desarrollador debe conocer el hardware y las redes? Esta unidad **abre el módulo 0483 (Sistemas informáticos)**, que da al desarrollador el conocimiento del **sistema** sobre el que corren sus aplicaciones: el **hardware**, el **sistema operativo** y la **red**. Aunque un desarrollador no monta ordenadores a diario, **necesita** entender el sistema para desarrollar, desplegar y depurar bien. Esta primera unidad cubre los **componentes** de un ordenador y sus **periféricos**, las **normas de seguridad**, y una **introducción a las redes** (tipos, componentes, topologías, cableado y el **mapa físico y lógico** de una red local). Culmina en el análisis de los componentes de un equipo y el esquema físico/lógico de una red local.

Desarrolla los **componentes y periféricos** (componentes del sistema; periféricos y adaptadores; normas de seguridad y PRL) y la **introducción a las redes** (características, tipos; componentes y topologías; cableado y conectores; mapa físico y lógico). El **análisis de los componentes de un equipo y el esquema físico/lógico de una red** es el entregable. Es qué componentes forman un sistema y cómo se conectan en red.



interior de un ordenador con sus componentes (placa base, CPU, RAM, almacenamiento)

SECCIÓN 1

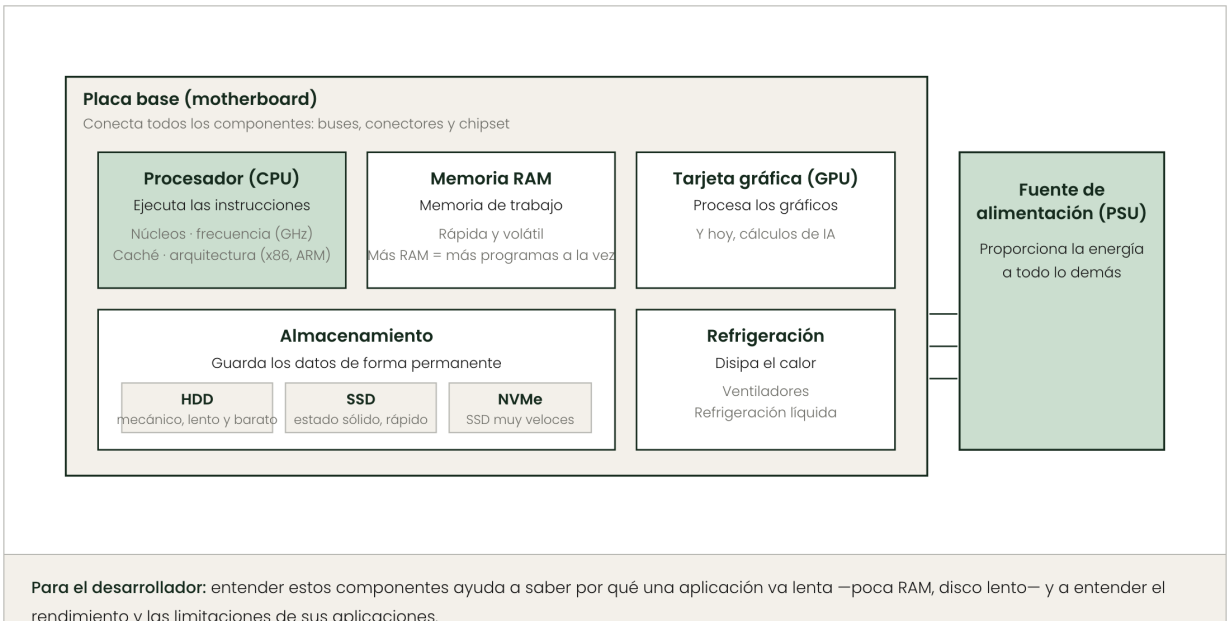
1. Componentes y periféricos

1.1. Componentes de un sistema informático

Un **sistema informático (ordenador)** está formado por varios **componentes de hardware** que colaboran para procesar la información. Conocerlos ayuda al desarrollador a entender el **rendimiento** y las **limitaciones** de sus aplicaciones:

- **La placa base (motherboard):** la **placa** que **conecta** todos los componentes; contiene los buses, los conectores y el chipset.
- **El procesador (CPU):** el «cerebro» que **ejecuta** las instrucciones. Sus características: **núcleos** (cores), **frecuencia** (GHz), **caché**, **arquitectura** (x86, ARM).
- **La memoria RAM:** la memoria **de trabajo** (rápida y volátil) donde se cargan los programas y datos en ejecución. Más RAM = más programas/datos a la vez.
- **El almacenamiento:** guarda los datos de forma **permanente**:
- **HDD** (disco duro mecánico, más lento y barato) y **SSD** (disco de estado sólido, rápido); los **NVMe** son SSD muy veloces.
- **La tarjeta gráfica (GPU):** procesa los **gráficos** (y hoy, cálculos de **IA**).
- **La fuente de alimentación (PSU):** proporciona la **energía**.
- **La refrigeración:** disipa el **calor** (ventiladores, refrigeración líquida).
- **Para el desarrollador:** entender estos componentes ayuda a saber por qué una aplicación va lenta (poca RAM, disco lento), a elegir un equipo de desarrollo adecuado y a optimizar el software. Comprender los **componentes** de un sistema informático (placa base, CPU, RAM, almacenamiento) es la base para evaluar un equipo. Comprender esto es el punto de partida. Es qué componentes forman un ordenador y qué hace cada uno.

Un sistema informático está formado por componentes de hardware que colaboran para procesar la información: la placa base (motherboard) conecta todos los componentes (buses, conectores, chipset); el procesador (CPU) ejecuta las instrucciones, con características como los núcleos, la frecuencia, la caché y la arquitectura (x86, ARM); la memoria RAM es la memoria de trabajo (rápida y volátil) donde se cargan los programas y datos en ejecución; el almacenamiento guarda los datos de forma permanente (HDD mecánico más lento y barato, SSD de estado sólido rápido, NVMe muy veloz); la tarjeta gráfica (GPU) procesa los gráficos y hoy los cálculos de IA; la fuente de alimentación (PSU) da la energía y la refrigeración disipa el calor; y para el desarrollador entender estos componentes ayuda a saber por qué una aplicación va lenta, elegir un equipo adecuado y optimizar el software.



componentes de un ordenador (placa base, CPU, RAM, almacenamiento, GPU, fuente)

1.2. Periféricos, adaptadores y seguridad

Además de los componentes internos, un sistema tiene **periféricos** (dispositivos externos) que se conectan mediante **adaptadores** e interfaces. Y trabajar con hardware exige unas **normas de seguridad**:

- **Periféricos:** dispositivos externos que amplían el sistema:
- **De entrada:** teclado, ratón, escáner, webcam, micrófono.
- **De salida:** monitor, impresora, altavoces.
- **De entrada/salida:** pantalla táctil, unidades de red.
- **Adaptadores y puertos de conexión:** cómo se conectan los periféricos:
- **USB** (el más universal, en versiones y con USB-C), **HDMI/DisplayPort** (vídeo), **Ethernet (RJ45)** (red), **audio (jack)**, **Bluetooth** y **Wi-Fi** (inalámbricos).
- **Normas de seguridad y prevención de riesgos laborales (PRL):** al manipular el hardware hay que seguir medidas:
- **Riesgo eléctrico:** desconectar y descargar la energía antes de abrir un equipo.
- **Electricidad estática (ESD):** descargarse (pulsera antiestática) para no dañar los componentes.
- **Riesgos físicos y ergonómicos:** manipulación de equipos pesados, postura ante la pantalla (0483 conecta con la salud laboral).
- **Gestión de residuos (RAEE):** los equipos viejos son residuos electrónicos que deben reciclarse correctamente. Comprender los **periféricos**, los **adaptadores** y las **normas de seguridad** completa el conocimiento del sistema físico. Comprender cómo se amplía y se manipula con seguridad un sistema es esencial.

Además de los componentes internos, un sistema tiene periféricos (dispositivos externos) conectados mediante adaptadores, y trabajar con hardware exige normas de seguridad: los periféricos son de entrada (teclado, ratón, escáner, webcam), de salida (monitor, impresora, altavoces) y de entrada/salida (pantalla táctil); los adaptadores y puertos de conexión son USB (el más universal, con USB-C), HDMI/DisplayPort (vídeo), Ethernet/RJ45 (red), audio (jack), Bluetooth y Wi-Fi (inalámbricos); y las normas de seguridad y PRL incluyen el riesgo eléctrico (desconectar antes de abrir un equipo), la electricidad estática/ESD (descargarse con pulsera

antiestática para no dañar los componentes), los riesgos físicos y ergonómicos (equipos pesados, postura ante la pantalla) y la gestión de residuos (RAEE, reciclar los equipos viejos correctamente).

Norma de seguridad y PRL	Qué exige
Riesgo eléctrico	Desconectar y descargar la energía antes de abrir un equipo
Electricidad estática (ESD)	Descargarse con pulsera antiestática para no dañar los componentes
Riesgos físicos y ergonómicos	Cuidado al manipular equipos pesados; postura correcta ante la pantalla
Gestión de residuos (RAEE)	Los equipos viejos son residuos electrónicos y deben reciclarse correctamente

Puerto o interfaz	Para qué
USB (con USB-C)	El más universal, para casi cualquier periférico
HDMI / DisplayPort	Vídeo
Ethernet (RJ45)	Red
Audio (jack)	Sonido
Bluetooth y Wi-Fi	Conexión inalámbrica

Tipo de periférico	Qué hace	Ejemplos
De entrada	Introduce información en el sistema	Teclado, ratón, escáner, webcam, micrófono
De salida	Presenta información al usuario	Monitor, impresora, altavoces
De entrada/salida	Ambas cosas	Pantalla táctil, unidades de red

Ejemplo — evaluar un equipo de desarrollo.



técnico analizando los componentes de un equipo informático

Contexto. Un desarrollador debe **evaluar** y elegir un **equipo de desarrollo** adecuado para su trabajo (compilar, ejecutar máquinas virtuales, contenedores), analizando sus componentes.

Estrategia. Analizó los **componentes** clave para el desarrollo: un **procesador** con varios núcleos (para compilar y virtualizar), **RAM** abundante (para el IDE, las VM y los contenedores a la vez), un **SSD NVMe** (para arrancar y compilar rápido) y una **GPU** adecuada. Consideró los **periféricos** (varios monitores) y las conexiones (USB-C, Ethernet). Valoró la **arquitectura** (x86 o ARM/Apple Silicon).

Resultado. El desarrollador **evaluó** el equipo entendiendo qué componente afecta a cada tarea, y eligió una configuración adecuada para desarrollar con fluidez. El caso muestra por qué conocer el hardware ayuda al desarrollador a elegir y optimizar su entorno.

En la práctica.

- Conoce los **componentes** (CPU, RAM, almacenamiento, GPU) y **qué** aporta cada uno al rendimiento.
- Para desarrollar, prioriza **RAM** abundante y **SSD/NVMe** (compilar, VM, contenedores).
- Usa las **conexiones** adecuadas (USB-C, HDMI, Ethernet) para los periféricos.
- Sigue las **normas de seguridad** al manipular hardware (riesgo eléctrico, ESD) y recicla los RAEE.

Un error frecuente es creer que «un desarrollador no necesita saber de hardware, solo de programación».

En realidad, conocer el hardware ayuda a entender el rendimiento y las limitaciones de las aplicaciones, a elegir el equipo adecuado y a depurar problemas (falta de memoria, disco lento). Es un conocimiento útil para el desarrollador.

Un error frecuente es creer que «da igual el tipo de almacenamiento que tenga el equipo».

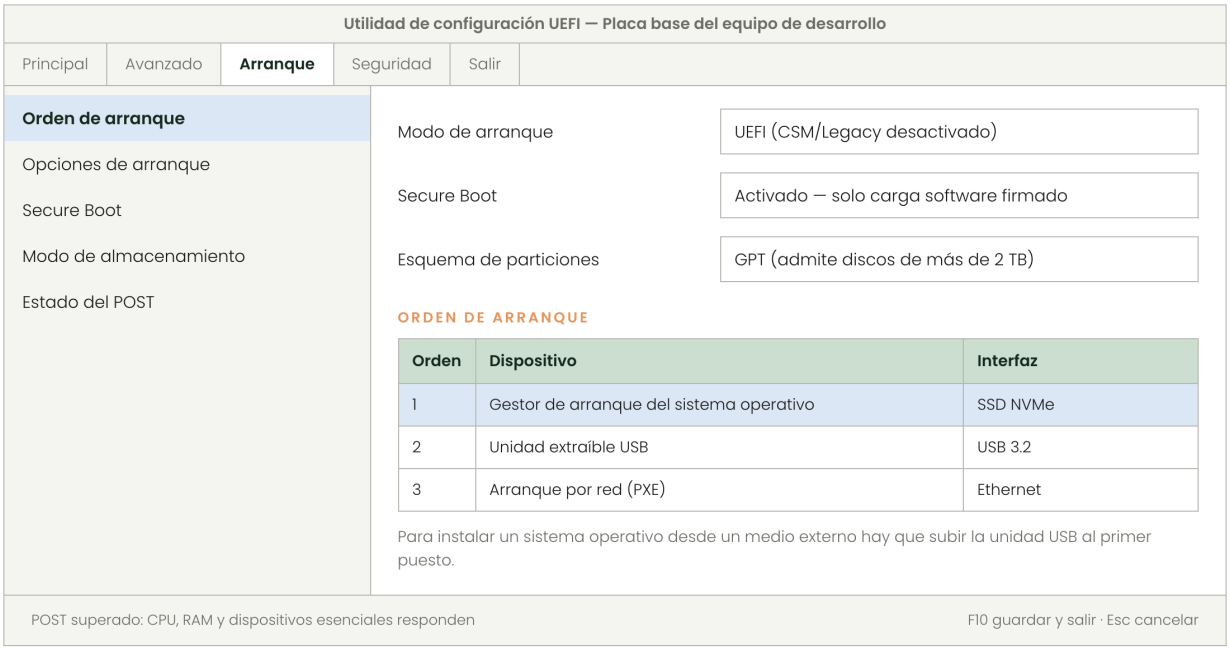
En realidad, el almacenamiento afecta mucho al rendimiento; un SSD/NVMe es muchísimo más rápido que un HDD mecánico para arrancar, compilar y trabajar. Para desarrollar, el SSD es prácticamente imprescindible.

1.3. El proceso de arranque: POST y BIOS/UEFI

Al encender el equipo, el firmware de la placa base ejecuta el POST (Power-On Self-Test): una autocomprobación que verifica que la CPU, la memoria RAM y los dispositivos esenciales responden antes de ceder el control al sistema operativo. Si algo falla, el POST lo avisa con pitidos o códigos en pantalla.

Ese firmware es hoy la UEFI (Unified Extensible Firmware Interface), que ha sustituido a la antigua BIOS. Frente a ella, la UEFI admite discos grandes con particiones GPT, ofrece interfaz gráfica, arranca más rápido e incorpora Secure Boot para impedir que se cargue software no firmado durante el arranque.

En la UEFI se configura el orden de arranque (qué dispositivo se intenta iniciar primero: disco, USB o red), un ajuste imprescindible para instalar un sistema operativo desde un medio externo.



pantalla de configuración UEFI con el orden de arranque y la opción Secure Boot

1.4. Estructura interna del procesador

Desde fuera, un procesador se resume en tres cifras: número de núcleos, frecuencia y tamaño de la caché. Por dentro es una máquina que repite un ciclo muy sencillo millones de veces por segundo, y conocer ese ciclo explica por qué dos programas que realizan el mismo número de operaciones pueden tardar tiempos muy distintos.

El punto de contacto entre el hardware y el software es el juego de instrucciones o ISA (Instruction Set Architecture): el repertorio de operaciones que la CPU entiende, los registros visibles al programa y la forma de direccionar la memoria. Un compilador traduce el código fuente a ese repertorio, y por eso un binario compilado para x86 no se ejecuta tal cual en ARM. En torno a ese repertorio se han consolidado dos enfoques:

- **CISC:** instrucciones numerosas y complejas, de longitud variable, capaces de hacer en una sola operación lo que en otro diseño ocuparía varias. Compacta el código y simplifica el trabajo del compilador, a costa de una decodificación complicada. La familia x86 es el ejemplo clásico.
- **RISC:** repertorio reducido, instrucciones de longitud fija y acceso a memoria limitado a operaciones de carga y almacenamiento. Decodificar resulta sencillo y la segmentación queda muy regular, a cambio de necesitar más instrucciones para la misma tarea. ARM y RISC-V siguen este enfoque. El compromiso entre ambos se ha ido difuminando: los procesadores x86 actuales traducen internamente sus instrucciones complejas a microoperaciones parecidas a las de un diseño RISC, y los diseños RISC han incorporado extensiones especializadas. Lo que sigue vigente es la consecuencia práctica: el binario depende de la arquitectura, y una aplicación multiplataforma se compila para cada una o se distribuye en un formato intermedio que se traduce en destino.

Dentro del procesador, los datos con los que se trabaja en cada instante viven en los registros, la memoria más rápida y más escasa del sistema:

- **Registros de propósito general:** contienen los operandos y los resultados intermedios de los cálculos.
- **Contador de programa (PC):** guarda la dirección de la siguiente instrucción. Al modificarlo se producen los saltos, las llamadas a función y los bucles.
- **Registro de instrucción (IR):** almacena la instrucción que se está decodificando y ejecutando en ese momento.

- **Registro de estado o de indicadores:** recoge banderas —cero, acarreo, signo, desbordamiento— que resumen el resultado de la última operación. Sobre ellas se apoyan los saltos condicionales en los que se traduce cualquier if o cualquier while.
- **Puntero de pila (SP):** apunta a la cima de la pila, donde se guardan las direcciones de retorno, los parámetros y las variables locales de cada llamada. Una recursión sin condición de parada agota esa zona y produce el desbordamiento de pila. La unidad aritmético-lógica (ALU) es la que calcula: suma, resta, compara y realiza las operaciones lógicas bit a bit. La unidad de control (UC) no calcula nada; interpreta la instrucción, genera las señales que activan al resto de bloques y decide qué ocurre en cada paso. Junto a ellas conviven la unidad de coma flotante, para los números reales, y las unidades vectoriales, que aplican una misma operación a varios datos a la vez y son la base del rendimiento en multimedia, cifrado y cálculo numérico.

El trabajo se organiza en el ciclo de instrucción, que se repite sin descanso: búsqueda de la instrucción en memoria a partir de la dirección que marca el contador de programa; decodificación, en la que la unidad de control averigua qué operación es y qué operandos necesita; ejecución en la unidad correspondiente; y escritura del resultado en un registro o en memoria. Al terminar, el contador ya apunta a la instrucción siguiente y todo vuelve a empezar.

Si cada instrucción esperase a que la anterior terminase por completo, la mayor parte del procesador estaría ociosa en cada etapa. La segmentación o pipeline divide el ciclo en fases y las solapa: mientras una instrucción se ejecuta, la siguiente se decodifica y una tercera se busca. El rendimiento sube mucho, pero aparecen los riesgos: dependencias de datos, cuando una instrucción necesita el resultado de otra que aún no ha terminado, y saltos cuyo destino todavía se desconoce.

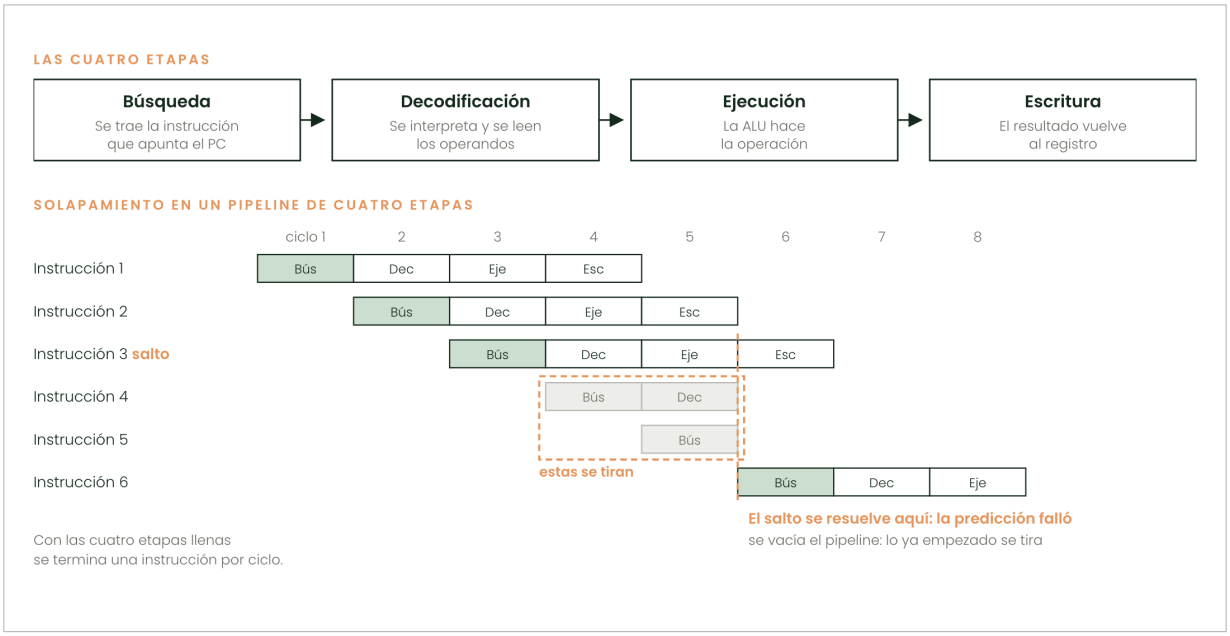
De ahí nacen la predicción de saltos y la ejecución especulativa, que adelantan trabajo suponiendo el camino más probable. Cuando la predicción falla hay que descartar lo adelantado y vaciar el pipeline, y ese descarte es tiempo perdido. La consecuencia para quien programa es medible: un bucle cuya condición es regular y predecible se ejecuta bastante más rápido que otro con la misma cantidad de instrucciones pero con saltos imprevisibles.

Entre los registros y la memoria principal se interpone la caché, memoria muy rápida integrada en el propio procesador y organizada en niveles. La L1 es minúscula, suele estar dividida en instrucciones y datos y es privada de cada núcleo; la L2 es mayor y algo más lenta, también por núcleo; y la L3 es la más grande y la comparten todos los núcleos. Los datos no se traen byte a byte, sino en líneas de tamaño fijo, de modo que al pedir un dato se trae también lo que hay a su lado.

Cuando el dato buscado está en la caché se produce un acierto; si no está, el fallo obliga a bajar al siguiente nivel y, en el peor caso, a la memoria principal, que es órdenes de magnitud más lenta. Por eso recorrer un vector de forma secuencial es mucho más rápido que saltar por una lista enlazada dispersa en memoria aunque tenga el mismo número de elementos: es el principio de localidad de referencia, espacial —lo que está cerca se usará pronto— y temporal —lo que se acaba de usar se volverá a usar—.

Un procesador multinúcleo integra varios núcleos completos que ejecutan instrucciones en paralelo de verdad. La multihebra simultánea, conocida comercialmente como hyper-threading, va un paso más allá y presenta cada núcleo como dos hilos lógicos que comparten sus unidades de ejecución: cuando uno se detiene esperando memoria, el otro aprovecha los huecos. La ganancia es real pero parcial, y conviene tenerlo presente al dimensionar un equipo o al fijar el grado de paralelismo de una compilación o de un servidor de aplicaciones: dos hilos lógicos no rinden como dos núcleos físicos.

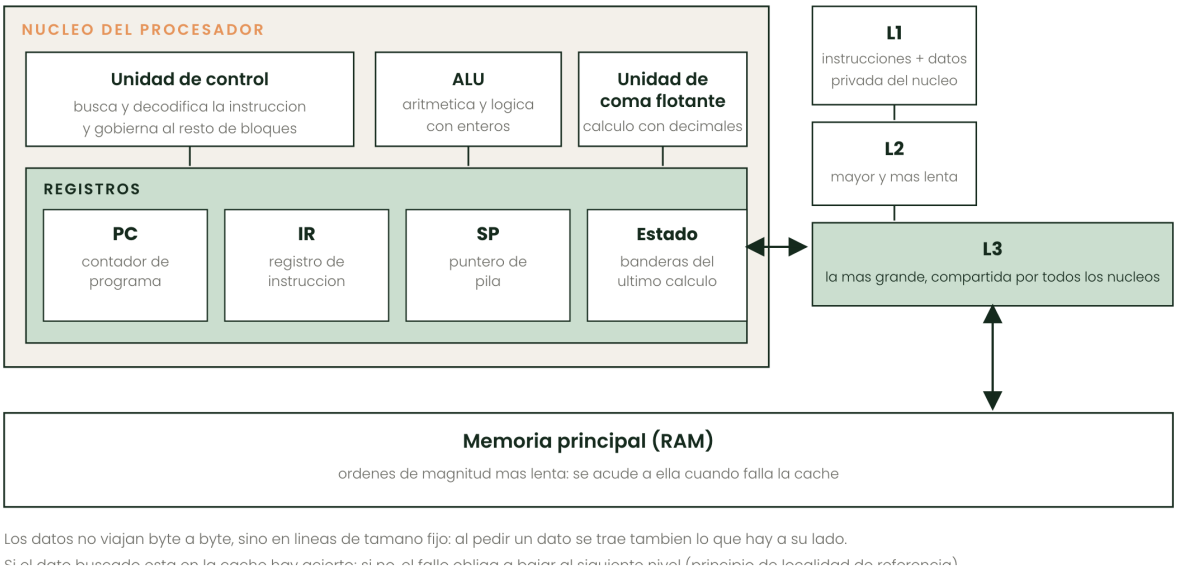
Todo esto se traduce en criterios concretos para quien desarrolla: preferir estructuras de datos contiguas cuando hay recorridos intensivos, evitar que varios hilos escriban en posiciones muy próximas porque acaban invalidándose la línea de caché mutuamente, no dar por hecho que añadir hilos multiplica el rendimiento —la parte que no se puede paralelizar marca el techo— y medir antes de optimizar, porque el cuello de botella rara vez está donde parece.



ciclo de instrucción (búsqueda, decodificación, ejecución y escritura) y su solapamiento en un pipeline de cuatro etapas, con el vaciado por fallo de predicción de salto

ARQUITECTURA INTERNA

Bloques del procesador y su conexión con los niveles de caché y la memoria principal



bloques internos del procesador (unidad de control, ALU, unidad de coma flotante y registros PC, IR, SP y de estado) y su conexión con los niveles de caché y la memoria principal

1.5. Interrupciones: la base de la multitarea y de la entrada/salida

Un procesador que se limitase a ejecutar instrucciones en orden sería incapaz de atender un teclado, una tarjeta de red o un disco sin quedarse esperando a cada uno. Las interrupciones son el mecanismo que resuelve ese problema y, aunque son invisibles desde el código, sostienen dos cosas que se dan por supuestas: que varios programas parezcan ejecutarse a la vez y que esperar a un dispositivo no consuma procesador.

Una interrupción es una señal que hace que el procesador suspenda de forma ordenada lo que está haciendo, guarde su estado, ejecute una rutina de atención y regrese exactamente al punto donde estaba, como si nada hubiera pasado. Según su origen se distinguen tres casos:

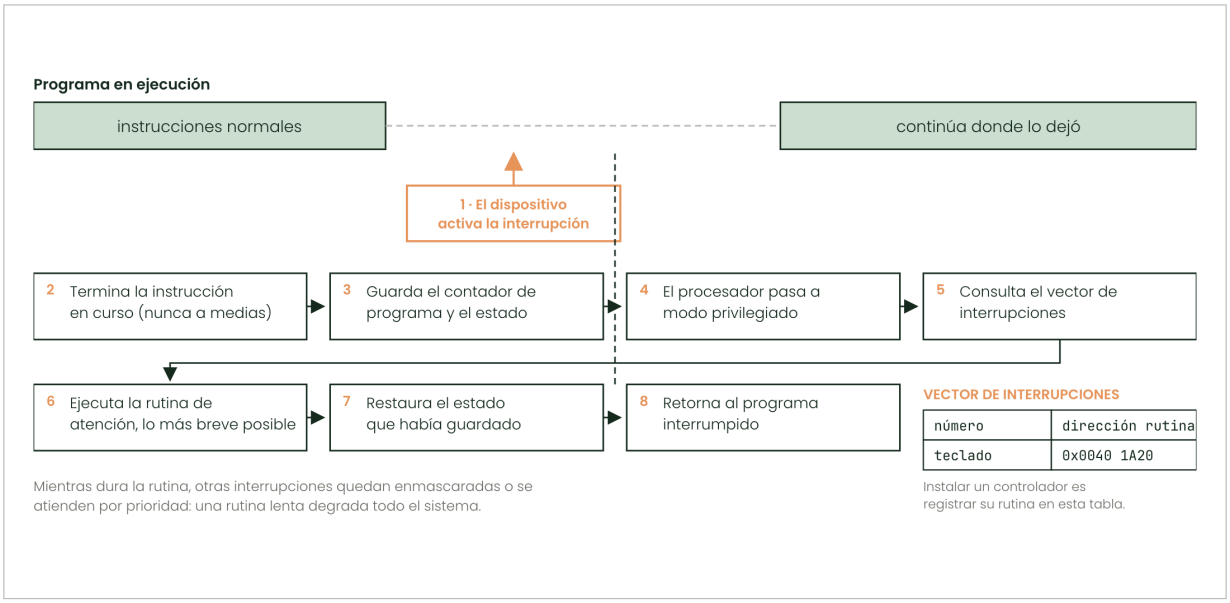
- **Interrupciones hardware:** las genera un dispositivo externo al procesador —el temporizador, el teclado, el adaptador de red, la controladora de disco— a través de un controlador de interrupciones que las recoge, las prioriza y las presenta a la CPU.
- **Interrupciones software:** las provoca el propio programa ejecutando una instrucción específica para solicitar un servicio al sistema operativo. Son la base de las llamadas al sistema, es decir, de la frontera entre las aplicaciones y el núcleo.
- **Excepciones:** las genera el procesador ante una situación anómala o especial durante la ejecución: división por cero, instrucción no válida, acceso a una dirección prohibida o fallo de página. El tratamiento es el mismo aunque el origen sea interno. Para saber qué código ejecutar en cada caso, el procesador consulta el vector de interrupción: una tabla en memoria, inicializada por el firmware y después por el sistema operativo, que asocia cada número de interrupción con la dirección de su rutina de atención. Instalar un controlador de dispositivo es, en el fondo, registrar en esa tabla la rutina que atenderá al hardware correspondiente.

La atención sigue siempre la misma secuencia: se termina la instrucción en curso, se guardan el contador de programa y el registro de estado, el procesador pasa a modo privilegiado, se ejecuta la rutina —que debe ser lo más breve posible, dejando el trabajo largo para después—, se restaura el estado guardado y se retorna. Mientras dura, otras interrupciones pueden quedar enmascaradas o ser atendidas según prioridad, y de ahí que una rutina lenta degrade la respuesta de todo el sistema.

De este mecanismo sale la multitarea. Un temporizador interrumpe periódicamente al procesador y devuelve el control al planificador del sistema operativo, que decide si el proceso en ejecución continúa o cede su turno a otro. Sin esa interrupción periódica, un programa que no cediera voluntariamente la CPU bloquearía el equipo entero; es exactamente la diferencia entre la multitarea cooperativa de los primeros sistemas personales y la multitarea con desalojo de los actuales.

Y de aquí sale también la eficiencia de la entrada/salida. La alternativa, el sondeo, consiste en preguntar una y otra vez al dispositivo si ya tiene el dato: es simple de programar y desperdicia todo el tiempo de procesador que dura la espera. Con interrupciones, el proceso que pide la operación queda bloqueado, el procesador se dedica a otra cosa y el dispositivo avisa cuando termina. Para las transferencias grandes se añade el acceso directo a memoria (DMA), que permite al dispositivo escribir directamente en la memoria e interrumpir una sola vez al finalizar.

Para quien desarrolla, esto explica un comportamiento cotidiano: una operación de entrada/salida no consume procesador mientras espera, consume tiempo. Distinguir un proceso limitado por CPU de otro limitado por entrada/salida cambia por completo la estrategia, porque en el primer caso ayuda repartir el trabajo entre núcleos y en el segundo lo que ayuda es solapar esperas, agrupar operaciones y reducir el número de accesos.



atención de una interrupción paso a paso: señal del dispositivo, fin de la instrucción en curso, guardado del estado, consulta del vector, ejecución de la rutina de atención y retorno al programa interrumpido

1.6. La memoria interna: tipos, características y jerarquía

Bajo el nombre de memoria interna conviven tecnologías muy distintas que se diferencian en si conservan o no la información al apagar, en cómo se graban y en para qué se usan. Distinguirlas es lo que permite entender por qué se puede ampliar la RAM pero no la caché, o por qué se puede actualizar la UEFI de una placa sin cambiar ningún chip.

La memoria volátil pierde su contenido en cuanto se corta la alimentación y es la memoria de trabajo del sistema:

- **Memoria dinámica (DRAM):** almacena cada bit como carga en un condensador diminuto que se descarga con el tiempo, por lo que necesita un refresco continuo. Es densa y económica, y es la que forma la memoria principal. Las variantes síncronas —SDRAM y sus sucesivas generaciones DDR— coordinan sus transferencias con el reloj del sistema y transmiten datos en los dos flancos de cada ciclo.
- **Memoria estática (SRAM):** guarda cada bit en un biestable y no necesita refresco. Es mucho más rápida, pero también mucho más cara y voluminosa por bit, así que se reserva para la caché integrada en el procesador. Ese es el motivo de que la caché se cuente en unos pocos megabytes y la RAM en decenas de gigabytes.
- **Módulos:** la RAM se presenta en módulos DIMM en sobremesa y servidores, y en SO-DIMM, más cortos, en portátiles y equipos compactos. En muchos portátiles y en los diseños con memoria unificada va soldada y no admite ampliación, algo que conviene comprobar antes de comprar y no después.
- **Canales:** el controlador de memoria puede trabajar con varios canales en paralelo. Instalar los módulos en los zócalos correctos y en parejas idénticas duplica el ancho de banda disponible; hacerlo mal deja el equipo funcionando en un solo canal, sin ningún aviso y con una pérdida de rendimiento que nadie relaciona con el montaje.
- **Corrección de errores (ECC):** los módulos con ECC detectan y corrigen fallos de un bit provocados por el ruido o la radiación. En un servidor evitan corrupciones silenciosas de datos; requieren soporte del procesador y de la placa, y por eso no se encuentran en cualquier equipo. Dos características se confunden con frecuencia. La latencia es el tiempo que transcurre desde que se pide un dato hasta que llega; el ancho de banda es la cantidad de datos por segundo que se pueden mover. Subir la frecuencia aumenta el ancho de banda, pero las latencias expresadas en ciclos suelen crecer también, de forma que el tiempo real de

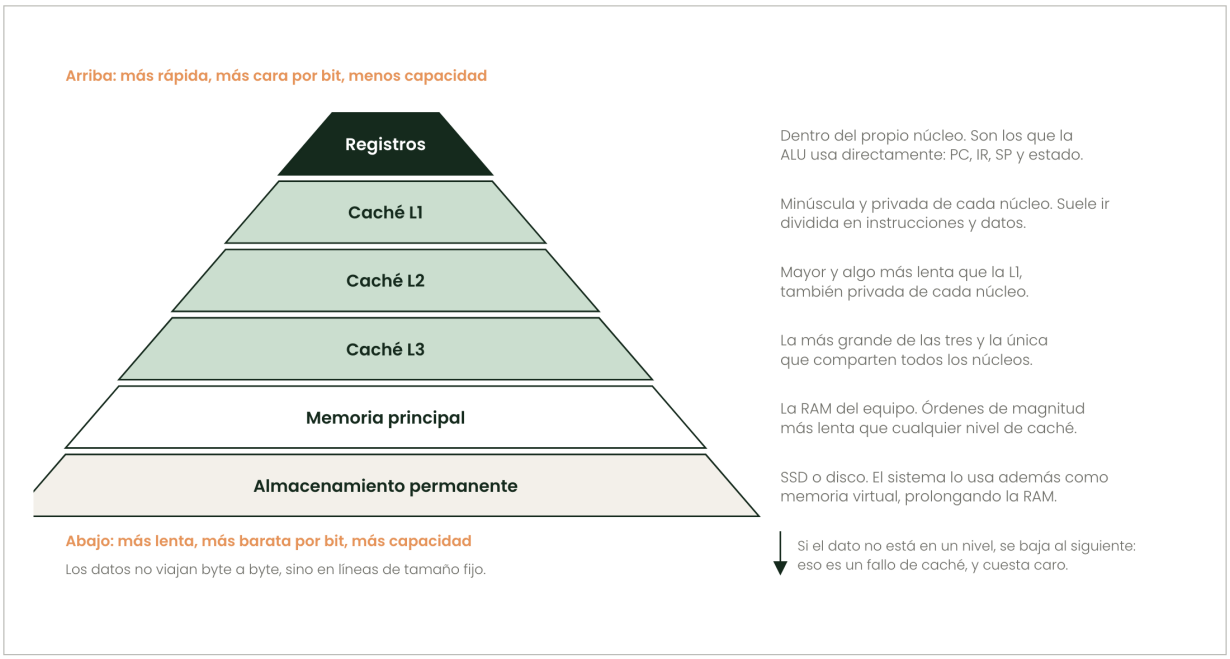
acceso apenas mejora. Una carga que recorre grandes volúmenes de datos agradece el ancho de banda; una carga con accesos dispersos y dependientes entre sí depende de la latencia.

Frente a ella, la memoria ROM conserva su contenido sin alimentación. Bajo ese nombre genérico conviven varias tecnologías, y lo que las diferencia es cómo se graban y cómo se borran:

- **ROM de máscara:** el contenido se define durante la fabricación del chip y no puede modificarse jamás. Solo compensa para tiradas muy grandes de un contenido definitivo.
- **PROM:** se fabrica virgen y se graba una sola vez de forma irreversible, fundiendo fusibles internos. Permitió que el contenido lo decidiese el fabricante del equipo y no el del chip.
- **EPROM:** se graba eléctricamente y se borra por completo exponiendo el chip a luz ultravioleta a través de una ventana de cuarzo. Era reutilizable, pero exigía extraerla del circuito para borrarla.
- **EEPROM:** se borra y se regraba eléctricamente, sin desmontar nada y con granularidad de byte. Es lenta y cara por bit, así que se emplea para pequeñas cantidades de datos de configuración y calibración.
- **Memoria flash:** evolución de la EEPROM que borra por bloques en lugar de byte a byte, lo que la hace mucho más rápida y mucho más densa. Es la que hoy guarda el firmware de la placa base y de los periféricos —de ahí que la UEFI pueda actualizarse— y la que forma las unidades SSD, las tarjetas de memoria y las memorias USB. La flash no es eterna: cada celda soporta un número limitado de ciclos de borrado y escritura, y por eso las unidades SSD llevan un controlador que reparte el desgaste entre todas las celdas y reserva bloques de repuesto. Esto explica dos hechos prácticos: que un SSD se degrade con la escritura y no con la lectura, y que una actualización de firmware interrumpida a mitad pueda dejar inservible una placa o una unidad.

Todo lo anterior encaja en la jerarquía de memoria, que ordena los soportes por velocidad, capacidad y coste: registros, caché en sus tres niveles, memoria principal y almacenamiento permanente. Al bajar por ella la capacidad crece y el precio por bit cae, pero el tiempo de acceso se dispara. El sistema operativo añade un escalón más con la memoria virtual, que utiliza el disco como prolongación de la RAM; funciona, pero cuando el sistema entra en intercambio continuo de páginas el rendimiento se hunde, y ese es el síntoma inconfundible de un equipo con memoria insuficiente para la carga que se le pide.

Tipo	Cómo se graba	Cómo se borra	Granularidad del borrado	Uso actual
ROM de máscara	En la propia fabricación del chip	No se borra	—	Solo compensa para tiradas muy grandes de un contenido definitivo
PROM	Una sola vez, fundiendo fusibles internos	Irreversible	—	Permitió que el contenido lo decidiera el fabricante del equipo y no el del chip
EPROM	Eléctricamente	Luz ultravioleta a través de una ventana de cuarzo	Todo el chip	Reutilizable, pero exigía extraerla del circuito
EEPROM	Eléctricamente	Eléctricamente, sin desmontar nada	**Byte**	Lenta y cara por bit: pequeñas cantidades de datos de configuración y calibración
Flash	Eléctricamente	Eléctricamente	**Bloque**	Firmware de la placa base y de los periféricos —de ahí que la UEFI pueda actualizarse—, unidades SSD, tarjetas de memoria y memorias USB



jerarquía de memoria en pirámide (registros, caché L1/L2/L3, memoria principal y almacenamiento) indicando cómo varían capacidad, velocidad y coste por bit al descender

1.7. La placa base: formatos, componentes y buses

La placa base fija lo que un equipo podrá llegar a ser: cuánta memoria admite, qué procesadores acepta, cuántas tarjetas y unidades caben dentro y qué caja y qué fuente le sirven. Es la pieza menos reversible del montaje, porque cambiarla obliga casi siempre a cambiar también el procesador y la memoria.

El factor de forma normaliza las dimensiones de la placa, la posición de los agujeros de fijación y la de la zona de conectores traseros, de modo que cualquier caja compatible sirva:

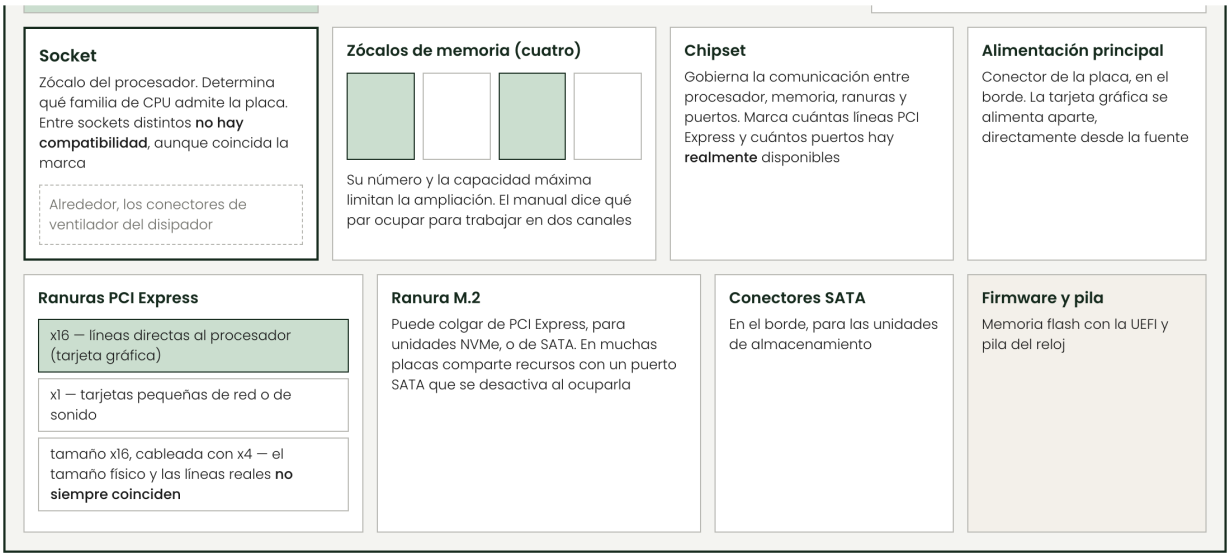
- **ATX:** el formato de referencia en sobremesa, de 305 × 244 mm. Es el que ofrece más ranuras de expansión y más zócalos de memoria y el que mejor se refrigera, a cambio de exigir una caja grande.
- **Micro-ATX:** de 244 × 244 mm, conserva la compatibilidad de fijación y de conectores con ATX y recorta ranuras de expansión. Es el equilibrio habitual en los equipos de oficina.
- **Mini-ITX:** de 170 × 170 mm, con una sola ranura de expansión y normalmente dos zócalos de memoria. Permite equipos muy compactos, pero obliga a vigilar la altura del disipador, la longitud de la tarjeta gráfica y la ventilación.
- **E-ATX:** más ancha que ATX, pensada para estaciones de trabajo y servidores con muchos zócalos de memoria, varias tarjetas de expansión y a veces dos procesadores. Requiere caja específica y suele complicar el cableado. Elegir formato no es elegir tamaño: condiciona la capacidad de ampliación futura, la potencia y el tipo de fuente que encaja, el número de unidades de almacenamiento que caben y la refrigeración que se puede montar. En los portátiles la placa es un diseño propietario del fabricante, con el procesador y a menudo la memoria soldados, lo que reduce la reparación a sustituir la placa completa; y en los equipos de formato reducido y los mini-PC se llega a integrar todo en una sola pieza, con la ampliación limitada al almacenamiento cuando la hay.

Sobre la placa, los componentes que determinan la compatibilidad y la ampliación son siempre los mismos:

- **Socket:** el zócalo del procesador. Determina qué familia de CPU admite la placa, y no existe compatibilidad entre sockets distintos por mucho que la marca coincida.

- **Chipset:** el conjunto de circuitos que gobierna la comunicación entre el procesador, la memoria, las ranuras y los puertos. Marca cuántas líneas PCI Express y cuántos puertos hay realmente disponibles, si se pueden combinar varias unidades y qué funciones de gestión y de arranque ofrece la placa.
- **Zócalos de memoria:** su número y la capacidad máxima admitida limitan la ampliación futura. Conviene comprobar la distribución de canales del manual antes de instalar los módulos.
- **Ranuras PCI Express:** conectan las tarjetas de expansión —gráfica, red, controladoras de almacenamiento—. Se describen por su tamaño físico y por el número de líneas que reciben en realidad (x1, x4, x8, x16), y ambos datos no siempre coinciden: una ranura del tamaño mayor puede estar cableada con muy pocas líneas.
- **Conectores SATA y ranuras M.2:** para las unidades de almacenamiento. Una ranura M.2 puede estar conectada a PCI Express, para unidades NVMe, o a SATA, y en muchas placas comparte recursos con un puerto SATA que se desactiva al ocuparla; es una fuente clásica de discos que desaparecen tras una ampliación.
- **Conectores de alimentación:** el conector principal de la placa y el específico del procesador, además de los de ventiladores. La tarjeta gráfica se alimenta directamente desde la fuente, y ese consumo es el que suele obligar a dimensionarla.
- **Firmware y batería:** la memoria flash que contiene la UEFI y una pila que mantiene el reloj y la configuración guardada. Una pila agotada se manifiesta como pérdida de la hora y del orden de arranque en cada apagado, y se confunde a menudo con un fallo del sistema operativo. Los buses son los caminos por los que circulan los datos entre todo lo anterior, y su ancho de banda depende del número de líneas y de la generación: cada generación de PCI Express duplica aproximadamente la velocidad por línea, de modo que una tarjeta puede rendir igual con la mitad de líneas en una generación más reciente. Ese razonamiento es el que permite decidir si conviene instalar una unidad NVMe en una ranura conectada directamente al procesador o en otra que cuelga del chipset y comparte camino con el resto de periféricos.

Formato	Dimensiones	Ranuras de expansión	Zócalos de memoria	Caja	Capacidad de ampliación
ATX	305 × 244 mm	Las más	Los más	Grande	La mayor; también la que mejor se refrigera
Micro-ATX	244 × 244 mm	Menos que ATX	Intermedios	Compatible con la de ATX en fijación y conectores	El equilibrio habitual en equipos de oficina
Mini-ITX	170 × 170 mm	**Una**	Normalmente dos	Muy compacta	Limitada: hay que vigilar la altura del disipador, la longitud de la gráfica y la ventilación
E-ATX	Más ancha que ATX	Varias	Muchos	Específica	Estaciones de trabajo y servidores, a veces con dos procesadores; complica el cableado



placa base de formato ATX identificando socket, chipset, zócalos de memoria, ranuras PCI Express, conectores SATA, ranura M.2, conectores de alimentación y chip de firmware

1.8. Interfaces de entrada/salida y criterios de elección

Un mismo periférico puede conectarse de varias formas y la elección no es indiferente: determina la velocidad real, el retardo, si el dispositivo se alimenta por el propio cable y cuántos aparatos van a competir por el mismo camino. Estas son las familias y su vía natural de conexión:

- **Almacenamiento interno:** SATA para unidades mecánicas y para SSD de formato clásico; PCI Express, a través de una ranura M.2 o de una ranura de expansión, para unidades NVMe, que evitan traducir a un protocolo pensado para discos giratorios y por eso ofrecen mucha más velocidad y muchas más operaciones simultáneas.
- **Almacenamiento externo:** USB en sus modos de alta velocidad para unidades portátiles, y conexiones de red para el almacenamiento compartido. El cuello de botella suele estar en el enlace, no en la unidad.
- **Vídeo:** HDMI y DisplayPort, ambos digitales y con audio integrado. DisplayPort permite encadenar varios monitores desde una sola salida y es la vía habitual de las configuraciones multipantalla; el conector USB-C puede transportar vídeo mediante modo alternativo, lo que explica que un mismo cable sirva para pantalla, datos y carga.
- **Periféricos de interacción:** USB para teclado, ratón, cámaras, lectores y dispositivos de captura; Bluetooth para los inalámbricos, con la contrapartida de un retardo mayor y de la necesidad de emparejamiento y de batería.
- **Red:** Ethernet con conector RJ45 cuando importan la estabilidad y la latencia, y Wi-Fi cuando importa la movilidad. Un puesto de desarrollo que compila contra repositorios remotos o que despliega imágenes grandes agradece el cable.
- **Adaptadores y concentradores:** un concentrador USB reparte un mismo enlace entre varios dispositivos, de modo que el ancho de banda se comparte y la suma de consumos puede superar lo que el puerto entrega; un adaptador de USB a Ethernet o a vídeo resuelve la falta de puertos, pero añade un traductor más en el camino y con él una fuente de fallos. Con USB conviene distinguir tres cosas que se confunden constantemente: el conector, es decir, la forma física —el tipo A rectangular o el tipo C reversible—; el protocolo y la velocidad que admite el enlace; y la capacidad de entregar energía. Dos cables con el mismo conector USB-C pueden diferir en si transportan vídeo, en la velocidad que soportan y en la potencia que

aceptan, y no hay forma de saberlo mirándolos. Cuando un monitor conectado por USB-C no da imagen o un portátil no carga, la causa suele estar ahí y no en el equipo.

Para elegir un interfaz sirven cuatro criterios. El ancho de banda necesario, que se estima a partir del volumen de datos y del tiempo aceptable. La latencia tolerable, que es lo que distingue un uso interactivo de una transferencia por lotes. La alimentación, porque un dispositivo que se alimenta por el cable evita una fuente externa pero limita el consumo posible. Y el número de dispositivos que van a compartir el enlace, ya que un bus compartido reparte lo que un enlace dedicado entrega íntegro.

A esos cuatro se añade un quinto criterio de mantenimiento: el interfaz más estándar y extendido siempre facilita el reemplazo, la compatibilidad con equipos futuros y el diagnóstico, porque permite sustituir el cable o el dispositivo por otro cualquiera para descartar componentes. Quien desarrolla se topa con estos límites más de lo que espera: al conectar un dispositivo físico para depurar una aplicación móvil, al usar una cámara o un lector desde una aplicación de escritorio, o al mover imágenes de máquinas virtuales entre equipos.

Criterio de elección	Qué decide
Ancho de banda necesario	Se estima a partir del volumen de datos y del tiempo aceptable
Latencia tolerable	Distingue un uso interactivo de una transferencia por lotes
Alimentación	Alimentarse por el cable evita una fuente externa pero limita el consumo posible
Número de dispositivos que comparten el enlace	Un bus compartido reparte lo que un enlace dedicado entrega íntegro
Estandarización	El interfaz más extendido facilita el reemplazo, la compatibilidad futura y el diagnóstico

Familia	Interfaz natural	Por qué
Almacenamiento interno	SATA · **PCI Express** (ranura M.2 o de expansión)	SATA para unidades mecánicas y SSD clásicos; PCI Express para NVMe, que evita traducir a un protocolo pensado para discos giratorios y da mucha más velocidad y más operaciones simultáneas
Almacenamiento externo	USB de alta velocidad · red	El cuello de botella suele estar en el enlace, no en la unidad
Vídeo	HDMI · DisplayPort	Ambos digitales y con audio integrado; DisplayPort permite encadenar varios monitores desde una sola salida
Red	Ethernet (RJ45) · Wi-Fi	Cable cuando importan la estabilidad y la latencia; inalámbrico cuando importa la movilidad
Concentradores y adaptadores	USB	Un concentrador reparte el mismo enlace y la suma de consumos puede superar lo que el puerto entrega; un adaptador añade un traductor más y con él una fuente de fallos

SECCIÓN 2

2. Introducción a las redes

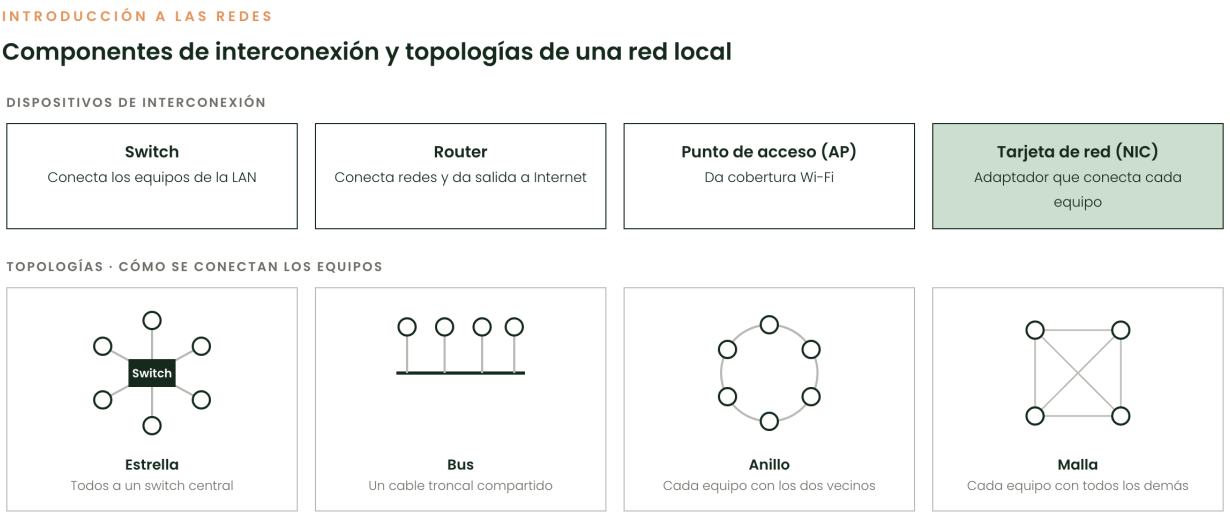
2.1. Características, tipos, componentes y topologías

Los sistemas informáticos rara vez trabajan aislados: se **conectan en red** para **compartir** recursos e información. El desarrollador necesita entender las **redes** porque sus aplicaciones se comunican a través de ellas:

- **Qué es una red:** un conjunto de **equipos conectados** que **comparten** recursos (archivos, impresoras, Internet) y se **comunican**.
- **Ventajas:** compartir recursos, comunicación, acceso centralizado, colaboración.
- **Tipos de redes según su alcance:**
- **LAN (Local Area Network):** red **local** (una oficina, un edificio).
- **WAN (Wide Area Network):** red **amplia** (la mayor es **Internet**).
- **MAN, PAN**, y las inalámbricas (**WLAN**, Wi-Fi).
- **Componentes de una red:**
- **Equipos** (nodos): ordenadores, servidores, dispositivos.
- **Dispositivos de interconexión: switch** (conecta los equipos de la LAN), **router** (conecta redes y da salida a Internet), **punto de acceso (AP)** (Wi-Fi).
- **Medios de transmisión:** cables o inalámbrico.
- **Tarjeta de red (NIC):** el adaptador que conecta cada equipo.
- **Topologías (cómo se conectan): estrella** (todos a un switch central, la más común), **bus**, **anillo**, **malla**.

Comprender las **características, tipos, componentes y topologías** de las redes es la base para entender cómo se conectan los sistemas. Comprender cómo se conectan los equipos entre sí en una red es esencial.

Los sistemas informáticos se conectan en red para compartir recursos e información, y el desarrollador necesita entenderlas porque sus aplicaciones se comunican a través de ellas: una red es un conjunto de equipos conectados que comparten recursos y se comunican, con ventajas como compartir recursos, comunicación, acceso centralizado y colaboración; los tipos según su alcance son la LAN (red local), la WAN (red amplia, la mayor Internet), la MAN, la PAN y las inalámbricas (WLAN, Wi-Fi); los componentes son los equipos/nodos, los dispositivos de interconexión (switch que conecta los equipos de la LAN, router que conecta redes y da salida a Internet, punto de acceso/AP para Wi-Fi), los medios de transmisión (cables o inalámbrico) y la tarjeta de red/NIC; y las topologías (cómo se conectan) son estrella (todos a un switch central, la más común), bus, anillo y malla.



componentes de una red (switch, router, AP) y topologías (estrella, bus, anillo, malla)

2.2. Cableado, conectores y mapa físico/lógico

Para montar una red hacen falta **medios de transmisión** (cables o inalámbrico) con sus **conectores**, y conviene documentarla con un **mapa físico y lógico**. Es lo que pide el entregable de la unidad:

- **Tipos de cableado y conectores:**
- **Par trenzado (UTP/STP):** el cable de red más común, con conector **RJ45**; en categorías (Cat 5e, Cat 6…).
- **Fibra óptica:** transmite luz, muy rápida y para largas distancias.
- **Coaxial** (en desuso para datos).
- **Inalámbrico (Wi-Fi):** sin cables, con sus estándares (Wi-Fi 5/6).
- **Direccionamiento (introducción):** cada equipo tiene una **dirección IP** que lo identifica en la red (se ve a fondo en el módulo de redes).
- **Mapa físico de la red:** representa **dónde** están y **cómo** se conectan **físicamente** los equipos y dispositivos (el cableado, los switches, la distribución en el edificio).
- **Mapa lógico de la red:** representa la **organización lógica**: las **direcciones IP**, las **subredes**, cómo fluyen los datos, independientemente de la ubicación física.
- **Por qué documentar:** el mapa físico y lógico permite **entender, mantener y ampliar** la red, y **diagnosticar** problemas. Comprender el **cableado**, los **conectores** y el **mapa físico/lógico** de una red completa la introducción a las redes y es el entregable. Comprender cómo se monta y se documenta una red local es esencial.

Para montar una red hacen falta medios de transmisión con sus conectores, y conviene documentarla con un mapa físico y lógico: los tipos de cableado y conectores son el par trenzado UTP/STP (el más común, con conector RJ45, en categorías Cat 5e/6), la fibra óptica (transmite luz, muy rápida y para largas distancias), el coaxial (en desuso para datos) y el inalámbrico/Wi-Fi (con sus estándares Wi-Fi 5/6); cada equipo tiene una dirección IP que lo identifica; el mapa físico representa dónde están y cómo se conectan físicamente los equipos (cableado, switches, distribución), y el mapa lógico representa la organización lógica (direcciones IP, subredes, flujo de datos) independientemente de la ubicación física; y se documenta para entender, mantener, ampliar y diagnosticar la red.

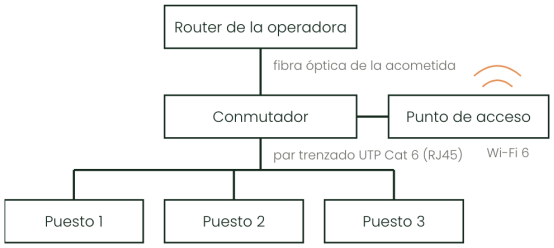
Ejemplo — componentes de un equipo y mapa de red (entregable).

COMPONENTES ANALIZADOS

Componente	Qué aporta al rendimiento	En el equipo analizado
CPU	Núcleos y frecuencia: compilación y tareas en paralelo	8 núcleos
RAM	Trabajar con VM y contenedores sin agotar la memoria	32 GB
Almacenamiento	Arranque, compilado y apertura de proyectos	SSD NVMe de 1 TB
GPU	Gráficos y cargas de cálculo paralelo	Integrada

Valores de ejemplo: el equipo concreto lo elige quien hace el entregable.

MAPA FÍSICO — DÓNDE ESTÁ Y CÓMO SE CONECTA



El coaxial está en desuso para datos.
Recoge el cableado y dónde está cada equipo.

MAPA LÓGICO — QUÉ DIRECCIÓN TIENE CADA UNO

Subred 192.168.1.0/24

Puerta de enlace	192.168.1.1
Puesto 1	192.168.1.10
Puesto 2	192.168.1.11
Puesto 3	192.168.1.12
Portátil por Wi-Fi	192.168.1.50

No depende de dónde esté físicamente cada equipo.
Direcciones de ejemplo.

análisis de los componentes de un equipo y esquema físico/lógico de una red local

Contexto. Como **entregable**, el desarrollador **analiza los componentes** de un equipo y elabora el **esquema físico y lógico** de una red local (por ejemplo, la de una pequeña oficina de desarrollo).

Estrategia. Analizó los componentes del equipo (CPU, RAM, almacenamiento, GPU, periféricos, conexiones) documentando sus características. Luego diseñó el **mapa físico** de la red (los equipos, el switch, el router, el cableado RJ45, el AP Wi-Fi, la distribución) y el **mapa lógico** (las direcciones IP, la subred, cómo salen a Internet por el router). Documentó ambos esquemas con claridad.

Resultado. El desarrollador entregó el **análisis** de los componentes de un equipo y el **esquema físico/lógico** de una red local, entendiendo el sistema completo (hardware + red). El análisis de los componentes de un equipo y el esquema físico/lógico de una red fueron el **entregable** de la unidad.

En la práctica.

- Las **redes** conectan los equipos para compartir recursos; tipos por alcance: **LAN**, **WAN** (Internet).
- Los componentes clave: **switch** (LAN), **router** (salida a Internet), **AP** (Wi-Fi), **tarjeta de red**.
- El **cableado** más común es el **par trenzado (RJ45)**; la **fibra** para velocidad y distancia.
- Documenta la red con su **mapa físico** (dónde y cómo se conecta) y **lógico** (IP, subredes).

Un error frecuente es creer que «para un desarrollador, las redes no son importantes».

En realidad, las aplicaciones se comunican por la red (con servidores, APIs, bases de datos); entender las redes ayuda a desarrollar, desplegar y depurar aplicaciones conectadas, que son casi todas hoy.

Un error frecuente es creer que «el mapa físico y el mapa lógico de una red son lo mismo».

En realidad, el mapa físico muestra dónde están y cómo se conectan físicamente los equipos (cables, distribución); el mapa lógico muestra la organización lógica (IP, subredes, flujo de datos). Son dos vistas complementarias.

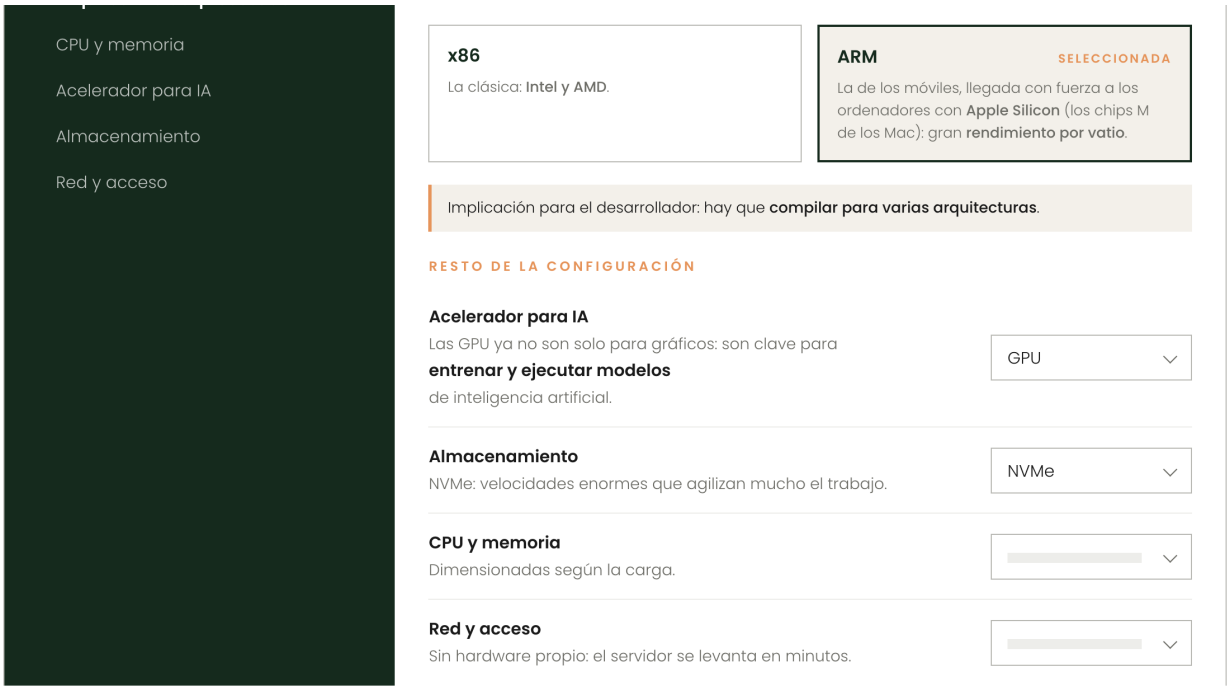
Ampliación y actualización

Arquitecturas actuales: ARM, GPU para IA y la nube como sistema

El hardware evoluciona rápido, y conviene conocer las **arquitecturas actuales**. En los procesadores, la arquitectura **ARM** (la de los móviles) ha llegado con fuerza a los ordenadores con el **Apple Silicon** (los chips M de los Mac), que ofrecen gran **rendimiento por vatio** frente a la clásica **x86** (Intel/AMD); esto tiene implicaciones para el desarrollador (compilar para varias arquitecturas). Las **GPU** ya no son solo para gráficos: son clave para los cálculos de **inteligencia artificial** (entrenar y ejecutar modelos). Y el **almacenamiento NVMe** ofrece velocidades enormes que agilizan mucho el trabajo.

Además, hoy el «sistema» sobre el que corre una aplicación no tiene por qué ser **físico**: la **nube** ofrece **máquinas virtuales** y recursos como **servicio (IaaS)** en proveedores como **AWS o Azure**. Un desarrollador puede levantar un servidor en la nube en minutos, sin hardware propio. Así, el conocimiento del «sistema» se extiende del hardware físico a la **infraestructura en la nube**. Para el desarrollador DAM, conocer las **arquitecturas actuales y la nube como sistema** lo mantiene al día. Refleja cómo el hardware evoluciona (ARM, GPU para IA, NVMe) y cómo la nube (IaaS) es hoy una alternativa al hardware físico.

El hardware evoluciona rápido y conviene conocer las arquitecturas actuales: en los procesadores, la arquitectura ARM (la de los móviles) ha llegado con fuerza a los ordenadores con el Apple Silicon (los chips M de los Mac), con gran rendimiento por vatio frente a la clásica x86 (Intel/AMD), con implicaciones para el desarrollador (compilar para varias arquitecturas); las GPU ya no son solo para gráficos sino clave para los cálculos de inteligencia artificial, y el almacenamiento NVMe ofrece velocidades enormes; además, hoy el sistema sobre el que corre una aplicación no tiene por qué ser físico, porque la nube ofrece máquinas virtuales y recursos como servicio (IaaS) en AWS o Azure, permitiendo levantar un servidor en minutos sin hardware propio, extendiendo el conocimiento del sistema del hardware a la infraestructura en la nube; conocerlas mantiene al día.



arquitecturas actuales (ARM/Apple Silicon vs x86, GPU para IA, NVMe) y la nube como sistema (IaaS)

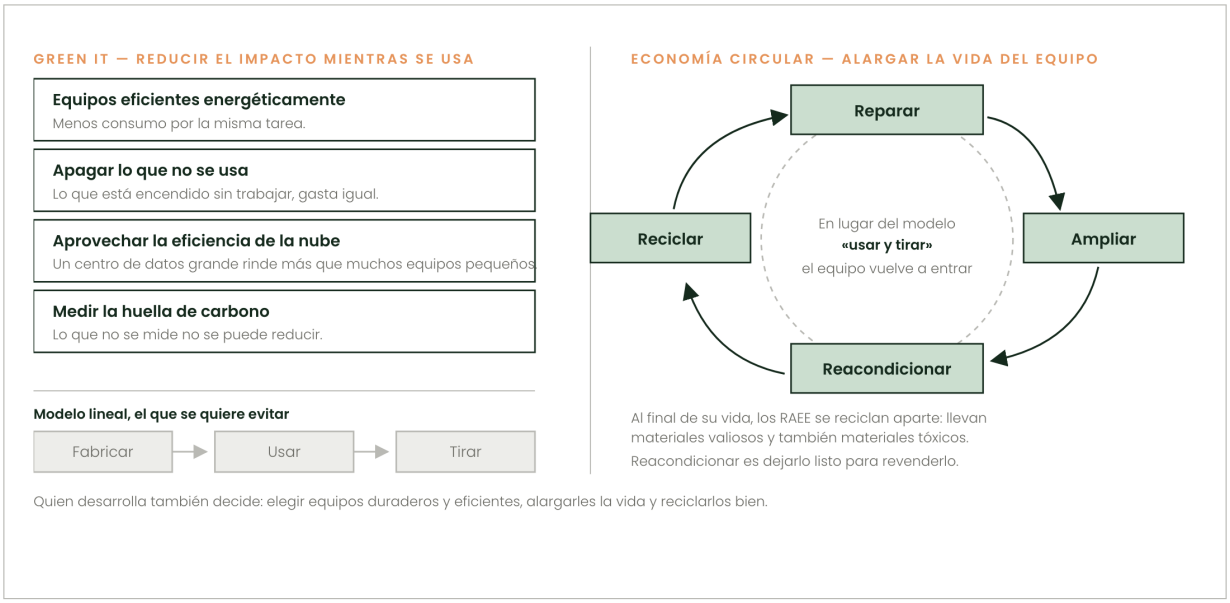
Sostenibilidad del hardware: green IT y economía circular

El hardware tiene un **impacto ambiental** importante (fabricación, consumo energético, residuos), así que la **sostenibilidad** es cada vez más relevante también en los sistemas informáticos. El **green IT (informática**

verde) busca reducir ese impacto: usar equipos **eficientes energéticamente**, apagar lo que no se usa, aprovechar la **eficiencia** de la nube (los grandes centros de datos son más eficientes que muchos equipos pequeños) y medir la **huella de carbono**.

La **economía circular** aplicada al hardware es otra pieza clave: en lugar del modelo «usar y tirar», se busca **alargar la vida** de los equipos (repararlos, ampliarlos, **reacondicionarlos** para revenderlos) y, al final de su vida, **reciclar** correctamente sus materiales (la gestión de los **RAEE**, con materiales valiosos y tóxicos). Un desarrollador o técnico contribuye eligiendo equipos duraderos y eficientes, alargando su vida y reciclándolos bien. Para el desarrollador DAM, conocer la **sostenibilidad del hardware (green IT, economía circular)** conecta la tecnología con la responsabilidad ambiental. Refleja cómo la sostenibilidad del hardware se aborda con el green IT (eficiencia) y la economía circular (reacondicionado, reciclaje de RAEE).

El hardware tiene un impacto ambiental importante (fabricación, consumo, residuos), por lo que la sostenibilidad es cada vez más relevante en los sistemas informáticos: el green IT (informática verde) busca reducir ese impacto usando equipos eficientes energéticamente, apagando lo que no se usa, aprovechando la eficiencia de la nube y midiendo la huella de carbono; la economía circular aplicada al hardware busca alargar la vida de los equipos (repararlos, ampliarlos, reacondicionarlos) y reciclar correctamente sus materiales al final de su vida (la gestión de los RAEE, con materiales valiosos y tóxicos), contribuyendo el desarrollador o técnico eligiendo equipos duraderos y eficientes, alargando su vida y reciclándolos bien; conocerla conecta la tecnología con la responsabilidad ambiental.



sostenibilidad del hardware (green IT, eficiencia energética) y economía circular (reacondicionado, RAEE)

Mini proyecto guiado

Componentes de un equipo y mapa de red. El sistema microinformático y sus componentes.

Paso 1: Analiza los componentes de un equipo (CPU, RAM, almacenamiento, GPU, periféricos) y documenta sus características.

Paso 2: Explica los **tipos** de redes, sus **componentes** (switch, router, AP) y su **topología**.

Paso 3: Diseña el **mapa físico** de una red local (equipos, switch, router, cableado, AP).

Paso 4: Diseña el **mapa lógico** (direcciones IP, subred, salida a Internet) y documéntalo.

Entregable: análisis de los componentes de un equipo y esquema físico/lógico de una red local.

Ejercicio

Analiza un sistema informático y su red.

Requisitos:

- Identifica y analiza los componentes de un equipo (CPU, RAM, almacenamiento, GPU, periféricos, conexiones).
- Aplica las normas de seguridad al manipular el hardware.
- Explica los tipos, componentes y topologías de las redes.
- Elabora el mapa físico y lógico de una red local.

Resumen profesionalizante

Con este conocimiento, ahora estás preparado para:

- Identificar los componentes de un sistema informático.
- Reconocer los periféricos y adaptadores de conexión.
- Aplicar las normas de seguridad y prevención de riesgos.
- Comprender las características, tipos y topologías de las redes.
- Elaborar el mapa físico y lógico de una red local.